Ecoole Supérieur de Technologie de Khénifra
Département : Génie de l'informatique et mathématiques
Filière : ingénierie de données et intelligence artificielle
Module : Apprentissage automatique

TP : k-means

Exercice1.

Objectifs

- Comprendre le fonctionnement de l'algorithme des k-means.
- Manipuler des données simples pour appliquer la classification non supervisée.
- Visualiser les étapes de regroupement.

Introduction théorique

- Expliquer brièvement le but de k-means : partitionner n points en k clusters.
- Rappeler la **fonction objectif** :

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

où Cj est le cluster j et μ_j son centroïde.

- Étapes principales :
    1. Choisir k centroïdes initiaux.
    2. Assigner chaque point au centroïde le plus proche.
    3. Recalculer les centroïdes comme la moyenne des points affectés.
    4. Répéter (2) et (3) jusqu'à convergence.

Données proposées

On considère un petit jeu de données **2D** pour pouvoir visualiser facilement :

Point	x	y
A	1	2
B	1	4
C	1	0
D	10	2
E	10	4
F	10	0

Objectif : appliquer k-means avec k=2

Travaux à faire

Partie 1 – Calculs à la main (papier/crayon)

1. Choisir 2 centroïdes initiaux (par ex. A et D).
2. Calculer la distance euclidienne de chaque point aux centroïdes.

$$d(x_i, \mu_j) = \sqrt{(x_i - \mu_j)^2 + (y_i - \mu_j)^2}$$

3. Affecter les points au cluster le plus proche.
4. Recalculer les centroïdes avec la formule :

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

5. Répéter jusqu'à convergence.

Partie 2 – Implémentation (Python)

- Utiliser sklearn.cluster.KMeans pour vérifier les résultats.
- Visualiser les clusters avec matplotlib.

4. Questions de réflexion

1. Que se passe-t-il si on choisit k=3 au lieu de k=2 ?
2. Pourquoi l'initialisation aléatoire peut donner des résultats différents ?
3. Quelle amélioration apporte **k-means++** ?

---

5. Extension : LE problème de la correspondance des classes

- Appliquer k-means sur un vrai dataset (par ex. **Iris dataset** en 2D).
- Comparer les clusters trouvés avec les classes réelles.

Exercice 2:

On souhaite créer un modèle K-Means pour regrouper (clustering) les clients qui ont des scores de dépenses presque similaires.

Nous allons utiliser le dataset Mall Customers.csv qui contient 200 enregistrements avec les attributs 'CustomerID', 'Genre', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)'.

1.) Importer les bibliothèques nécessaires : numpy, pandas, matplotlib et sklearn. La bibliothèque sklearn comprend la fonction suivante:

- from sklearn.cluster import KMeans

2.) Importer le dataset Mall Customers.csv .

3.) Visualiser les 5 premières et 5 dernières lignes du dataset.

4.) Afficher les dimensions du dataset.

5.) Préciser les données d'entrée X, correspondant aux colonnes 'Annual Income (k$)' et 'Spending Score (1-100)'.

6.) Afficher les données d'entrée X.

Nous allons utiliser la méthode Elbow pour trouver la valeur optimale de K.

7.) Ecrire une fonction elbow() qui calcule les valeurs de wcss pour des valeurs de K allant de 1 à10. Utiliser la fonction 'KMeans()', et fit(). Préciser les paramètres de la fonction KMeans() suivants: n clusters = k, init = 'k-means++', max iter =300, n init = 10, random state = 0. Utiliser le paramètre inertia pour retrouver la valeur de wcss. Faire l'appel de la fonction elbow() pour afficher les valeurs de wcss pour chaque valeur de K.

8.) Visualiser le graphe des valeurs de wcss en fonction de la valeur de K. Utiliser les fonctions plot(), title(), xtable(), ytable(), et show().

9.) Choisir la valeur optimale de K en analysant la courbe d'Elbow.

10.) Créer un modèle K-Means avec la valeur optimale de K. Faire l'apprentissage du modèle avec la fonction fit() et la prédiction sur les données d'entrée X avec la fonction predict(). Afficher les résultats de prédiction.

11.) Essayer cette fois-ci d'utiliser la fonction fit _predict() pour faire l'apprentissage du modèle K-Means et la prédiction avec les données X. Afficher les résultats de prédiction. Qu'est ce que vous remarquez ?

12.) Visualiser les point de données sous forme de cluster en utilisant la fonction scatter(). Utiliser les paramètres suivants: X[Y Kmeans == clusteri, 0], X[Y Kmeans == clusteri,1],s = 100, c='red', label = 'clusteri'. La variable clusteri prend les valeurs 0, 1, 2, 3 et 4.

13.) Visualiser les centroids aussi. Utiliser la fonction scatter() avec les paramètres suivants: kmeans.cluster centers [:,0], kmeans.cluster centers [:,1], s = 300, c = 'yellow', label = 'Centroids'.

14.) Utiliser les fonctions legend(), title(), xlabel(), ylabel() et show() pour visualiser la figure.